



PROYECTO 02 · DATALOGGER WEB

Logger WiFi

El instrumento crea su propia red, sirve su propia app y atiende a varios a la vez. **El instrumento es el servidor.**

H O Y

Mismo logger, otra topología

1 • 15 min

¿Qué cambia desde
BLE?

2 • 25 min

Modo Access Point

3 • 20 min

HTTP, JSON y seq

4 • 15 min

La página vive en la
flash

5 • 45 min

Manos a la obra

6 • 30 min

Experimentos y
discusión

Lo único nuevo es el enlace

	01 • Bluetooth LE	02 • WiFi (Access Point)
Topología	punto a punto, un solo cliente	red propia, varios clientes a la vez
Entrega	push (notificación)	pull (polling 1/s)
Formato	binario con especificación	texto JSON autodescriptivo
Navegadores	Chrome/Edge — no iOS	todos, celular incluido
La app vive en...	internet (GitHub Pages)	el propio ESP32

El instrumento ES la red

publica

SSID propio

EL831-G3: el ESP32 aparece en la lista de redes como cualquier router.

reparte

DHCP

le da una dirección IP a cada cliente que se conecta, sin configurar nada.

atiende

192.168.4.1

el instrumento vive siempre en la misma dirección de su propia red.

Hasta ahora el ESP32 se colgaba de una red ajena (modo **estación**). En modo **AP** no depende de la red del laboratorio ni de internet: es la arquitectura de los instrumentos de campo configurables por WiFi.

¿Por qué la app no está en internet?

https

Página en Pages

contexto seguro, candadito



http

fetch a 192.168.4.1

bloqueado: *mixed content* — una página segura no puede pedir a un origen inseguro

solución

El ESP32 sirve su propia página

página y datos salen del **mismo origen** — el problema desaparece, y queda una idea linda: el "deploy" de la app es compilar el firmware.

Preguntar una vez por segundo

```
GET /medicion                                ← el navegador pregunta (polling)
200 OK · application/json
{"seq":123, "temp_c":24.31, "pote_pct":57.5}    ← el instrumento responde
```

seq igual

misma medición — se descarta, el sensor mide 1/s

seq +1

muestra nueva — va al registro

seq salta

muestras perdidas — el hueco queda contado

◆ En BLE preguntábamos "¿cómo detectar una pérdida?". Acá la respuesta vino en el payload: **un número de secuencia**. Texto legible + contador: la filosofía opuesta al binario con spec.

El ciclo de la app embebida

1 • editar

pagina.html

un HTML común, dentro del proyecto



2 • probar

?demo=1

doble click: anda en el navegador, sin ESP32



3 • compilar

PlatformIO la embebe solo

`board_build.embed_txtfiles`
la mete en la flash (~16 kB) —
sin SD, sin filesystem, sin scripts

No hay paso de "deploy" de la app: **el build es el deploy.**

Firmware: una red y dos rutas

```
WiFi.softAP("EL831-G3", "el831logger"); // la clave: 8+ caracteres

servidor.on("/", atenderRaiz); // sirve la página embebida
servidor.on("/medicion", atenderMedicion); // sirve el JSON con seq

// loop(): medir 1/s SIN delay(), para no frenar al servidor
servidor.handleClient();
if (millis() ≥ proximaMedicionMs) {
  proximaMedicionMs = millis() + 1000;
  g_seq++;
  g_temp_c = medirTemperaturaC();
  g_pote_pct = medirPotePct(); // canal 2: ADC1 (34, o 3 en C3)
}
```


La app: polling con memoria

```
const r = await fetch('/medicion', { cache: 'no-store' });
const j = await r.json();

if (j.seq !== ultimoSeq) {                                // ¿es una muestra nueva?
  if (j.seq > ultimoSeq + 1)
    console.warn('muestras salteadas:', j.seq - ultimoSeq - 1);
  ultimoSeq = j.seq;
  agregarMuestra(j.temp_c, j.pote_pct);    // timestamp: lo pone el navegador
}
```

El resto de la página es **idéntico al proyecto 01**: mismo gráfico, mismas estadísticas, mismo CSV. Cambió el enlace; el logger ni se enteró.

Cuatro pasos y registrando

1

Cablear el DS18B20 (GPIO 4 + pull-up) y el pote (GPIO 34 • en C3: GPIO 3, ADC1); cambiar AP_SSID por el del grupo.

2

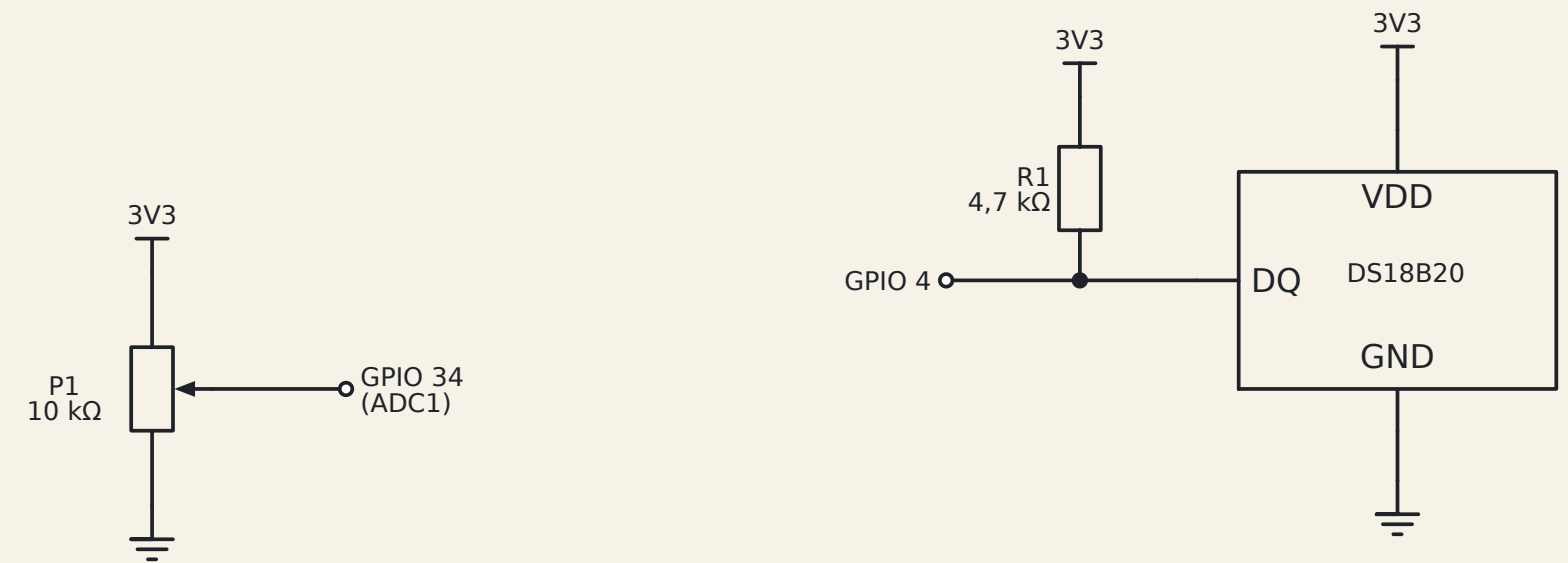
PlatformIO → **subir** el proyecto esp32_wifi.

3

Conectarse a la red del grupo desde el celular o la PC (clave el831logger).

4

Abrir <http://192.168.4.1> — el registro arranca solo.



El celular va a avisar "red sin internet": **mantener la conexión** (o apagar datos móviles).

Para hacer en el banco

a

Dos celulares a la vez

cada navegador arma SU registro. Comparar los CSV: mismo seq, distinto timestamp. ¿Por qué?

b

Pausar y reanudar

al volver, seq saltó: el sensor nunca paró. ¿Dónde quedaron esas muestras?

c

Medir el alcance

¿dónde corta WiFi? ¿Más o menos lejos que BLE? Anotar con qué obstáculos.

d

Bajar el período de polling

a 500 y a 2000 ms en la página: mirar qué pasa con seq en cada caso.

¿Dónde debería vivir el registro?

hoy

En el navegador

simple, cero infraestructura — pero muere con la pestaña y cada cliente tiene SU copia.

alternativa

En el instrumento

tarjeta SD: sobrevive a todo, pero hay que ir a buscarlo y administrar el espacio.

alternativa

En un servidor

registro único y compartido, acceso remoto... acá empieza la **telemedida**.

- ◆ No hay respuesta correcta: hay **requisitos**. ¿Cuánto vale perder una muestra? ¿Quién consume el dato y desde dónde? El transporte y el registro se eligen según el uso.

Errores típicos

La red no aparece

clave de menos de 8 caracteres: `softAP()` falla en silencio. El Serial Monitor lo delata.

El celular se desconecta solo

por "red sin internet" — mantener conexión o apagar los datos móviles.

192.168.4.1 no responde

seguís conectado a la red de siempre, no a la del ESP32.

Quisieron servirla desde Pages

fetch bloqueado: *mixed content*. Es la trampa que justifica todo el diseño — dejala fallar una vez.

La tabla que hay que poder defender

	01 • Bluetooth LE	02 • WiFi (AP)
¿Quién inicia?	el navegador elige el dispositivo	el navegador pide por HTTP
Entrega	push • notify	pull • polling con seq
Clientes	uno	varios a la vez
Alcance típico	~10 m	~30–50 m
Consumo	muy bajo	medio/alto
Pérdidas	¿cómo detectarlas?	seq las cuenta
Infraestructura	app en internet (HTTPS)	ninguna: el instrumento sirve todo

PROYECTO 02 · LOGGER WIFI

Mismo sensor, misma app, otro enlace

La comunicación es una **decisión de diseño**, no un accesorio.

Código y guías: **github.com/fernandorvs/Datalogger-Web-ESP32-2026**